Exp 1 : Write the Python Program to solve 8-Puzzle Problem.

Aim:

To write a python program to solve the 8-puzzle problem.

Algorithm:

Step1: Dequeue the state with the lowest priority from the frontier.

Step2: If the dequeued state is the goal state, return the solution path.

Step3: Add the dequeued state to the explored set.

Step4: Generate all possible successor states by swapping the empty tile with its neighbouring tiles.

Step5: For each successor state, calculate its priority based on the Manhattan distance and add it to the frontier if it has not been explored before.

Program:

```
from collections import deque

# Goal state we want to reach
goal = [[1, 2, 3],
        [5, 8, 6],
        [0, 7, 4]]

# Starting state (change this to your own puzzle)
start = [[1, 2, 3],
         [5, 6, 0],
         [7, 8, 4]]

def find_blank(state):
    """Step 1: locate the position (row, col) of the blank (0)"""
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def print_state(state):
    """Step 2: print a 3x3 grid the same way as the screenshot"""
```

```
    for row in state:
        print(" ".join(str(x) for x in row))
    print()


def get_neighbors(state):
    """Step 3: generate all boards reachable by sliding the blank
    up, down, left, or right"""
    neighbors = []
    x, y = find_blank(state)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # up, down, left, right

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]  # deep copy
            new_state[x][y], new_state[nx][ny] =
new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)
    return neighbors


def solve(start, goal):
    """Step 4: BFS search — explore level by level until goal is
found,
    keeping track of the path taken to reach each state"""
    start_t = tuple(map(tuple, start))
    goal_t = tuple(map(tuple, goal))

    queue = deque([(start_t, [start_t])])
    visited = {start_t}

    while queue:
        current, path = queue.popleft()

        if current == goal_t:
```

```
            return path  # list of states from start to goal

        current_list = [list(row) for row in current]
        for neighbor in get_neighbors(current_list):
            neighbor_t = tuple(map(tuple, neighbor))
            if neighbor_t not in visited:
                visited.add(neighbor_t)
                queue.append((neighbor_t, path + [neighbor_t]))

    return None  # no solution found

# Step 5: run the solver and print each intermediate board
solution_path = solve(start, goal)

if solution_path:
    for state in solution_path:
        print_state(state)
    print(f"Result: Hence, the simple python program for 8-
puzzle is done in {len(solution_path)-1} moves")
else:
    print("No solution found")
```

Output :

```
============================== RESTART: D:/test.py ==============================
1 2 3
5 6 0
7 8 4

1 2 3
5 0 6
7 8 4

1 2 3
5 8 6
7 0 4

1 2 3
5 8 6
0 7 4

Result: Hence, the simple python program for 8-puzzle is done in 3 moves
```

Result: Hence, the simple python program for 8- puzzle is done